LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

11

Linux Text Processing and Command-Line Power Tools

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 Introduction
02 Pipes and Redirection
03 grep: Search and Filter
04 awk: Extract Columns
05 sed: Modify Text
06 cut and tr
07 sort, uniq and wc
08 xargs
09 tee, paste and column
10 diff and cmp
11 jq: Processing JSON
12 Nginx Log Analysis
13 Command Cheat Sheet
14 Troubleshooting Examples
15 Best Practices
16 Interview Q&A
17 Takeaways and Exercises

Ravi Tomar

Linux · DevOps · Cloud Infrastructure

Part 1 · Core Linux

github.com/rv-ops/linux-interview-preparation-guide

CHAPTER OBJECTIVE

Learn how to search, filter, extract, transform and compare text from the command line using `grep`, `awk`, `sed`, `sort`, `uniq`, `xargs`, `jq` and related tools, and combine them into pipelines for log analysis and production troubleshooting.

1 Introduction

In DevOps, you work with logs, configuration files, command output, CSV files and JSON responses every day.

You don't always need to open a file or write a script to analyze it. Linux provides command-line tools that search, filter, extract, transform and combine data. For example, you may need to:

- Find all failed requests in Nginx logs.
- Extract client IP addresses from access logs.
- Identify the most frequent HTTP status codes.
- Search application logs for errors.
- Extract specific columns from a CSV file.
- Read JSON responses from an API.
- Compare two configuration files.
- Process thousands of files automatically.

Basic `grep`, `find` and `tail` usage was covered in Chapter 2. This chapter goes deeper and focuses on combining tools.

2 Pipes and Redirection

Pipe (|)

A pipe sends the output of one command as input to the next command.

```
ps aux | grep nginx
```

1. `ps aux` displays running processes.
2. `grep nginx` filters the output.
3. Only matching lines are displayed.

```
# Show filesystem usage, excluding tmpfs entries
df -h | grep -v tmpfs
```

INTERVIEW TRAP

A pipe carries only standard output, not errors. If a command prints errors to stderr, `cmd | grep error` will not see them. Use `cmd 2>&1 | grep error`.

Output Redirection

Operator	Purpose
`>`	Write output to a file, replacing existing content
`>>`	Append output to a file
`2>`	Redirect error output
`2>>`	Append error output
`&>`	Redirect both output and errors (Bash)
`2>&1`	Send errors to the same place as normal output

```
# Save command output
ls -l > files.txt

# Append a message to a log
echo "Deployment started" >> deployment.log

# Save normal output and errors in the same file
./deploy.sh > deploy.log 2>&1
```

`>` overwrites existing content; use `>>` to append. For the redirection order trap, see Chapter 10.

3 grep – Search and Filter Text

`grep` searches for matching text or patterns in files and command output.

Common Options

Option	Purpose
`-i`	Ignore uppercase/lowercase differences
`-n`	Show line numbers
`-v`	Show non-matching lines
`-r`	Search recursively
`-w`	Match whole words
`-c`	Count matching lines
`-l`	Show filenames containing matches
`-o`	Print only the matching part
`-A 3` / `-B 3` / `-C 3`	Show 3 lines after / before / around a match
`-E`	Use extended regular expressions
`-F`	Treat the pattern as plain text, not a regex

Practical Examples

```
# Search for errors
grep "ERROR" application.log

# Case-insensitive search with line numbers
grep -in "failed" application.log

# Search recursively
grep -rn "database connection failed" /var/log/

# Search for multiple patterns
grep -E "ERROR|WARN|CRITICAL" application.log

# Show context around an error
grep -C 3 "OutOfMemoryError" application.log

# Count matching lines
grep -c "ERROR" application.log

# Find files containing a configuration value
grep -rl "server_name" /etc/nginx/
```

Regular Expressions

Pattern	Meaning
`^ERROR`	Lines starting with ERROR
`failed$`	Lines ending with failed
`^[0-9]`	Lines starting with a digit
`error|failed`	Either error or failed (with `-E`)
`[0-9]+`	One or more digits (with `-E`)
`.`	Any single character

```
# HTTP 5xx responses in an Nginx access log
# (the status code follows the closing quote of the request)
grep -E '" 5[0-9]{2} ' access.log
```

INTERVIEW TRAP

`grep -v "200" access.log` does not reliably remove successful requests. It removes every line containing "200" anywhere, including IPs, URLs and byte counts. Match the status field instead: `awk '$9 != 200' access.log`.

DEVOPS USE CASE

When a deployment fails, search application logs for `ERROR`, `Exception` or `failed` instead of reading the entire log.

4 awk – Extract and Process Columns

`awk` is a powerful tool for structured text. By default, it treats spaces and tabs as field separators.

Sample file `employees.txt`:

```
101 Ravi DevOps 35000
102 Amit Developer 40000
103 Neha Tester 32000
```

Variable	Meaning
`$0`	Entire line
`$1`, `$2` ...	First column, second column and so on
`NF`	Number of fields in the line (`$NF` = last field)
`NR`	Current line number

```
# First column
awk '{print $1}' employees.txt

# Multiple columns
awk '{print $1, $3}' employees.txt

# Last column
awk '{print $NF}' employees.txt

# Line numbers
awk '{print NR, $0}' employees.txt
```

Conditions

```
# Employees earning more than 35000
awk '$4 > 35000 {print $2, $4}' employees.txt

# Only the DevOps team
awk '$3 == "DevOps" {print $2}' employees.txt

# Total salary
awk '{sum += $4} END {print "Total:", sum}' employees.txt
```

Custom Delimiters

```
# CSV data
awk -F ',' '{print $2, $3}' employees.csv

# /etc/passwd: username and login shell
awk -F ':' '{print $1, $7}' /etc/passwd
```

INTERVIEW TRAP

Shell variables are not expanded inside awk's single quotes. Pass them using `-v` : `awk -v limit="$LIMIT" '$4 > limit' employees.txt` .

5 sed – Modify and Transform Text

`sed` is a stream editor used to search, replace, delete and modify text without opening the file. By default, it prints the result and does not change the file.

Replace Text

```
# Replace the first "old" on each line
sed 's/old/new/' file.txt

# Replace all occurrences on each line
sed 's/old/new/g' file.txt

# Use a different delimiter when the text contains slashes
sed 's|/var/www/old|/var/www/new|g' nginx.conf
```

Modify a File Directly

```
# Modify the original file
sed -i 's/localhost/127.0.0.1/g' config.txt

# Safer: create config.txt.bak before modifying
sed -i.bak 's/localhost/127.0.0.1/g' config.txt
```

Delete and Print Lines

```
# Delete lines containing DEBUG
sed '/DEBUG/d' application.log

# Delete empty lines (including lines with only spaces)
sed '/^[[:space:]]*$/d' file.txt

# Delete line 5
sed '5d' file.txt

# Print only lines 10 to 20
sed -n '10,20p' application.log
```

WARNING

Be careful with `sed -i` on production configuration files. Preview the change without `-i`, keep a backup, and validate the configuration (for example, `nginx -t`) before reloading the service.

6 cut and tr

cut – Extract Fields

Sample file:

```
Ravi:DevOps:35000
Amit:Developer:40000
Neha:Tester:32000
```

```
# First field
cut -d ':' -f1 employees.txt

# First and third fields
cut -d ':' -f1,3 employees.txt

# Characters 1 to 5
cut -c1-5 file.txt
```

`-d` sets the delimiter, `-f` selects fields and `-c` selects character positions.

CUT VS AWK

`cut` is simple and fast for basic field extraction. `awk` supports conditions and calculations, and handles multiple spaces between columns. That is why `cut` gives wrong results on `ps` or `df` output.

tr – Translate or Delete Characters

```
# Lowercase to uppercase
echo "linux devops" | tr 'a-z' 'A-Z'

# Remove digits
echo "server123" | tr -d '0-9'

# Replace spaces with underscores
echo "linux server admin" | tr ' ' '_'

# Squeeze repeated spaces
echo "Linux     DevOps" | tr -s ' '

# Remove Windows line endings
tr -d '\r' < windows.txt > unix.txt
```

7 sort, uniq and wc

sort

```
# Alphabetical
sort names.txt

# Numeric, and reverse numeric
sort -n numbers.txt
sort -nr numbers.txt

# Sort by the fourth column, numerically
sort -k4,4n employees.txt

# Human-readable sizes (largest directories last)
du -sh /var/* | sort -h

# Sort and remove duplicates
sort -u names.txt
```

uniq

`uniq` finds or removes duplicate lines, but only when they are **next to each other**.

Command	Purpose
`uniq -c`	Count repeated lines
`uniq -d`	Show only duplicate lines
`uniq -u`	Show only unique lines

```
# Count how many times each line appears anywhere in the file
sort application.log | uniq -c
```

> INTERVIEW LINE
>
> **Always sort before uniq.**
>
> The common "most frequent" pipeline is: `extract | sort | uniq -c | sort -nr | head`.

wc

Command	Purpose
`wc -l`	Count lines
`wc -w`	Count words
`wc -c`	Count bytes
`wc -m`	Count characters

```
# Count error lines
grep -i "error" application.log | wc -l

# Count files in a directory
find /var/log -type f | wc -l
```

INTERVIEW TRAP

`grep -c` counts matching **lines**, not occurrences. A line with two errors is counted once. To count every occurrence, use `grep -o "ERROR" file | wc -l`.

8 xargs – Pass Input as Command Arguments

`xargs` takes input from standard input and passes it as arguments to another command. It is needed for commands such as `rm` and `cp`, which do not read filenames from stdin.

Option	Purpose
`-n 1`	Pass one argument at a time
`-I {}`	Replace a placeholder with each input item
`-0`	Read null-separated input
`-r`	Do not run the command when input is empty (GNU)

```
# Remove matching temp files safely, even with spaces in names
find /tmp -name "*.tmp" -print0 | xargs -0 rm -f --

# One ping per server
xargs -n 1 ping -c 1 < servers.txt

# Copy each listed file to /backup
xargs -I {} cp -- {} /backup/ < files.txt
```

INTERVIEW TRAP

`cat files.txt | xargs rm` splits names on spaces, so `my report.txt` becomes two arguments and the wrong file may be deleted. Use `find -print0 | xargs -0` for filenames.

9 tee, paste and column

tee – Display and Save Output Together

```
# Show output and save it
df -h | tee disk_usage.txt

# Append instead of overwrite
df -h | tee -a disk_usage.txt

# Watch a deployment while saving its log
./deploy.sh 2>&1 | tee deployment.log
```

INTERVIEW TRAP

The exit code of `./deploy.sh | tee deployment.log` is tee's, so a failed deployment can look successful in CI. Use `set -o pipefail` (see Chapter 10) or check `${PIPESTATUS[0]}`.

paste – Combine Lines from Files

```
names.txt      roles.txt
Ravi           DevOps
Amit           Developer
Neha           Tester
```

```
# Join side by side
paste names.txt roles.txt

# Use a comma separator
paste -d ',' names.txt roles.txt
```

column – Align Text into Columns

```
column -t employees.txt

# For CSV data
column -s ',' -t employees.csv
```

10 diff and cmp – Compare Files

```
# Line-by-line differences
diff config_old.conf config_new.conf

# Unified format (most readable)
diff -u config_old.conf config_new.conf

# Byte-level comparison
cmp file1.txt file2.txt
```

In unified format, lines starting with `-` were removed and lines starting with `+` were added.

> **DEVOPS USE CASE**
>
> Compare configuration before and after a deployment, or production against staging. To compare the same file on two servers: `diff <(ssh web1 cat /etc/nginx/nginx.conf) <(ssh web2 cat /etc/nginx/nginx.conf)` .

11 jq – Process JSON Data

API responses, cloud CLI output and Kubernetes resources use JSON. `jq` is a command-line JSON processor.

```
# Install
sudo apt install jq      # Ubuntu / Debian
sudo dnf install jq      # RHEL-based
```

Sample `app.json`:

```
{
  "name": "application",
  "environment": "production",
  "status": "running",
  "server": { "name": "web01", "ip": "192.168.1.10" }
}
```

```
# Pretty-print
jq '.' app.json

# Extract a value
jq -r '.name' app.json

# Multiple values
jq -r '.name, .environment, .status' app.json

# Nested value
jq -r '.server.ip' app.json
```

`-r` prints plain text without JSON quotes; use it when the value goes into a script.

Filtering Arrays

```
{
  "servers": [
    {"name": "web01", "status": "running"},
    {"name": "web02", "status": "stopped"},
    {"name": "web03", "status": "running"}
  ]
}
```

```
# Running servers only
jq -r '.servers[] | select(.status == "running") | .name' servers.json
```

AWS and Kubernetes Examples

```
# EC2 instance IDs and states
aws ec2 describe-instances --output json \
  | jq -r '.Reservations[].Instances[] | [.InstanceId, .State.Name] | @tsv'

# Kubernetes pod names and phases
kubectl get pods -o json | jq -r '.items[] | [.metadata.name, .status.phase] | @tsv'
```

> **DEVOPS NOTE**
>
> Use `jq` for JSON instead of `grep` or `sed`. Text matching breaks easily when JSON formatting changes.

12 Practical Nginx Log Analysis

Sample `access.log`:

```
192.168.1.10 - - [05/Nov/2026:10:10:01 +0000] "GET / HTTP/1.1" 200 1024
192.168.1.11 - - [05/Nov/2026:10:10:02 +0000] "GET /login HTTP/1.1" 500 512
192.168.1.10 - - [05/Nov/2026:10:10:03 +0000] "POST /api HTTP/1.1" 200 2048
192.168.1.12 - - [05/Nov/2026:10:10:04 +0000] "GET /dashboard HTTP/1.1" 404 256
192.168.1.11 - - [05/Nov/2026:10:10:05 +0000] "GET /api HTTP/1.1" 502 128
```

In this format, `$1` is the client IP, `$7` is the URL path and `$9` is the HTTP status code. Field positions depend on the Nginx `log_format`, so check a few lines first.

Failed Requests (400 or Higher)

```
awk '$9 ⩾ 400' access.log
```

Only 5xx Errors

```
awk '$9 ⩾ 500 && $9 < 600' access.log
```

Count HTTP Status Codes

```
awk '{print $9}' access.log | sort | uniq -c | sort -nr
```

```
      2 200
      1 502
      1 500
      1 404
```

Top 5 Client IPs

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -5
```

1. Extract the IP addresses.

2. Sort them so identical IPs are together.
3. Count each IP with `uniq -c`.
4. Sort by count in descending order and show the top 5.

Total Requests

```
wc -l < access.log
```

Monitor Errors in Real Time

```
tail -F /var/log/nginx/error.log | grep --line-buffered -i "error"
```

`--line-buffered` makes matches appear immediately in a pipeline. `-F` keeps following the file after log rotation.

13 Command Cheat Sheet

Requirement	Command
Find errors in logs	`grep -i "error" app.log`
Count error lines	`grep -ic "error" app.log`
Find HTTP 5xx requests	`awk '$9 ≥ 500 && $9 < 600' access.log`
Find top IP addresses	`awk '{print $1}' access.log | sort | uniq -c | sort -nr`
Find large files	`find /var/log -type f -size +100M`
Count files	`find /var/log -type f | wc -l`
Compare configurations	`diff -u old.conf new.conf`
Extract JSON values	`jq -r '.key' file.json`
Save command output	`command | tee output.log`
Remove empty lines	`sed '/^[[:space:]]*$/d' file.txt`
Extract a CSV column	`cut -d ',' -f2 file.csv`
Find duplicate lines	`sort file.txt | uniq -c`

14 Production Troubleshooting Examples

Example 1 Application is returning HTTP 500

Identify the affected requests:

```
awk '$9 == 500' /var/log/nginx/access.log
```

Then inspect the application logs:

```
grep -inE "exception|error|failed" /var/log/application.log
```

Check the same time window and correlate the application error with the Nginx request.

Example 2 Identify which IP is sending too many requests

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head -10
```

High request counts alone do not prove malicious activity. It may be a load balancer, a monitoring check or a shared NAT IP. Investigate the request pattern first.

Example 3 Find which deployment changed a configuration

```
diff -u application.conf.backup application.conf
```

Review the differences before deciding whether to roll back.

Example 4 **Extract failed services from a status report**

Suppose `service_status.txt` contains:

```
nginx running
mysql running
redis failed
docker running
```

```
awk '$2 == "failed" {print $1}' service_status.txt
```

```
redis
```

15 Best Practices

1. **Use pipes:** combine small commands instead of processing large files manually.
2. **Understand the input format:** `awk` and `cut` depend on the correct field delimiter.
3. **Match fields, not whole lines:** `awk '$9 == 500'` is more reliable than `grep 500`.
4. **Quote filenames** to protect paths containing spaces.
5. **Back up before in-place edits** with `sed -i.bak`.
6. **Use `jq` for JSON**, not text matching.
7. **Handle filenames safely** with `find -print0` and `xargs -0`.
8. **Validate before acting:** preview results before deleting files or changing configuration.

16 Common Interview Questions

Q1. **What is grep used for?**

`grep` searches for text or patterns in files and command output. It is commonly used for log analysis and troubleshooting.

Q2. **What is the difference between grep and awk?**

`grep` searches and filters matching lines. `awk` works with fields and supports conditions and calculations.

Q3. What is the difference between sed and awk?

`sed` is mainly used for substitution, deletion and stream editing. `awk` is better for processing fields and calculations.

Q4. What is the difference between cut and awk?

`cut` extracts simple fields or character positions. `awk` supports conditions, calculations and multiple spaces between columns.

Q5. Why do you sort before uniq?

`uniq` only detects duplicates that are next to each other. Sorting groups identical lines together first.

Q6. What does xargs do?

`xargs` converts input from standard input into command arguments, for example to pass filenames to `rm` or `cp`.

Q7. What is the purpose of tee?

`tee` displays command output and writes it to a file at the same time.

Q8. How do you find the top 10 IP addresses in an Nginx access log?

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -10
```

Q9. How do you count error lines in an application log?

```
grep -ic "error" application.log
```

This counts lines. To count every occurrence, use `grep -oi "error" application.log | wc -l`.

Q10. What is jq used for?

`jq` parses, filters and extracts data from JSON. It is commonly used with AWS CLI, Kubernetes and REST APIs.

Q11. How do you compare two configuration files?

```
diff -u old.conf new.conf
```

Q12. What is the difference between `>` and `>>`?

`>` overwrites a file. `>>` appends to the existing content.

Q13. Why does `cmd | grep error` sometimes miss errors shown on screen?

A pipe carries only standard output. Errors written to stderr bypass it. Use `cmd 2>&1 | grep error`.

17 Chapter Takeaways

After completing this chapter, you should be comfortable with:

- ✓ Pipes, redirection, and why stderr bypasses pipes.
- ✓ Searching and filtering text using `grep` and regular expressions.
- ✓ Extracting fields using `awk` and `cut`.
- ✓ Modifying text using `sed` and `tr`.
- ✓ Sorting and counting data using `sort`, `uniq` and `wc`.
- ✓ Passing input to commands safely using `xargs`.
- ✓ Saving and displaying output using `tee`.
- ✓ Comparing files using `diff` and `cmp`.
- ✓ Processing JSON using `jq`.
- ✓ Building pipelines to analyze production logs.

PRACTICAL EXERCISES

Use sample files if you do not have access to production logs.

1. Create a sample log with INFO, WARNING and ERROR messages. Display only ERROR lines.
2. Display the error lines with line numbers.
3. Count ERROR lines with `grep -c`, then count every occurrence with `grep -o | wc -l`.
4. Extract the first column from a sample employee file using `awk`.
5. Replace `localhost` with `127.0.0.1` in a test configuration file using `sed -i.bak`.
6. Sort a list of server names alphabetically.
7. Find duplicate IP addresses in a file.
8. Count HTTP status codes in an Nginx access log.
9. Find the top five client IP addresses in an access log.
10. Extract Kubernetes pod names and statuses using `jq`.
11. Compare two versions of an Nginx configuration file using `diff -u`.
12. Run a command and save its output using `tee`.

INTERVIEW TARGET

Be able to explain grep, awk, sed, pipes, redirection and jq, and solve a basic Nginx log-analysis problem without referring to notes.

NEXT CHAPTER

Chapter 12: Linux Logging, Monitoring and Scheduling

You will learn where Linux stores logs, how to use journalctl and logrotate, how to monitor CPU, memory, disk and network health, and how to schedule tasks using cron, at and systemd timers.

END OF CHAPTER 11